

customer_analysis.py

```
1 """
2 Customer Data Analysis & Insights
3 -----
4 Loads raw customer data, cleans and preprocesses it, runs exploratory
5 analysis, and outputs summary visualizations and a bottleneck report.
6
7 Usage:
8     python customer_analysis.py
9 """
10
11 import pandas as pd
12 import numpy as np
13 import matplotlib.pyplot as plt
14
15 INPUT_FILE = "customer_data.csv"
16 OUTPUT_DIR = "outputs"
17
18
19 def load_data(path: str) -> pd.DataFrame:
20     """Load raw customer data from CSV."""
21     df = pd.read_csv(path, parse_dates=["signup_date"])
22     print(f"Loaded {len(df)} rows, {df.shape[1]} columns.")
23     return df
24
25
26 def clean_data(df: pd.DataFrame) -> pd.DataFrame:
27     """Clean and preprocess the raw dataset.
28
29     - Removes exact duplicate rows
30     - Fills missing numeric values with the column median
31     - Fills missing categorical values with 'Unknown'
32     - Drops rows with invalid customer IDs
33     """
34     before = len(df)
35     df = df.drop_duplicates()
36     print(f"Removed {before - len(df)} duplicate rows.")
37
38     df = df[df["customer_id"].notna()]
39
40     numeric_cols = df.select_dtypes(include=[np.number]).columns
41     for col in numeric_cols:
42         missing = df[col].isna().sum()
43         if missing:
44             df[col] = df[col].fillna(df[col].median())
45             print(f"Filled {missing} missing values in '{col}' with median.")
46
47     categorical_cols = df.select_dtypes(include=["object"]).columns
48     for col in categorical_cols:
49         missing = df[col].isna().sum()
50         if missing:
51             df[col] = df[col].fillna("Unknown")
52             print(f"Filled {missing} missing values in '{col}' with 'Unknown'.")
```

```

53
54     return df.reset_index(drop=True)
55
56
57 def add_features(df: pd.DataFrame) -> pd.DataFrame:
58     """Engineer a few simple features used in the analysis."""
59     df["tenure_days"] = (pd.Timestamp.today() - df["signup_date"]).dt.days
60     df["is_active"] = df["orders_last_90d"] > 0
61     df["ticket_rate"] = df["support_tickets"] / df["orders_last_90d"].replace(0, 1)
62     return df
63
64
65 def exploratory_summary(df: pd.DataFrame) -> None:
66     """Print key summary statistics and trends."""
67     print("\n--- Spend by Segment ---")
68     print(df.groupby("segment")["monthly_spend"].agg(["mean", "median",
69 "count"]).round(2))
70
71     print("\n--- Spend by Region ---")
72     print(df.groupby("region")
73 ["monthly_spend"].mean().round(2).sort_values(ascending=False))
74
75     print("\n--- Satisfaction vs Support Tickets (correlation) ---")
76     corr = df["satisfaction_score"].corr(df["support_tickets"])
77     print(f"Correlation: {corr:.2f}")
78
79     print("\n--- Inactive Customers (0 orders in last 90 days) ---")
80     inactive_pct = (~df["is_active"]).mean() * 100
81     print(f"{inactive_pct:.1f}% of customers had no orders in the last 90 days.")
82
83
84 def identify_bottlenecks(df: pd.DataFrame) -> pd.DataFrame:
85     """Flag segments/regions with high support-ticket rates or low satisfaction,
86 which is where operational bottlenecks tend to show up."""
87     summary = (
88         df.groupby(["region", "segment"])
89         .agg(
90             avg_satisfaction=("satisfaction_score", "mean"),
91             avg_ticket_rate=("ticket_rate", "mean"),
92             customers=("customer_id", "count"),
93         )
94         .round(2)
95         .reset_index()
96     )
97     bottlenecks = summary[
98         (summary["avg_satisfaction"] < summary["avg_satisfaction"].median())
99         & (summary["avg_ticket_rate"] > summary["avg_ticket_rate"].median())
100     ].sort_values("avg_ticket_rate", ascending=False)
101
102     print("\n--- Potential Bottleneck Segments (low satisfaction, high ticket rate) ---")
103     print(bottlenecks.to_string(index=False))
104     return bottlenecks
105
106
107 def make_visualizations(df: pd.DataFrame, outdir: str) -> None:

```

```

        """Save a couple of key charts to the output folder."""
108     import os
109     os.makedirs(outdir, exist_ok=True)
110
111     plt.figure(figsize=(7, 4))
112     df.groupby("segment")["monthly_spend"].mean().plot(kind="bar", color="#1F3864")
113     plt.title("Average Monthly Spend by Segment")
114     plt.ylabel("Avg Monthly Spend")
115     plt.tight_layout()
116     plt.savefig(f"{outdir}/spend_by_segment.png")
117     plt.close()
118
119     plt.figure(figsize=(7, 4))
120     df.groupby("region")["satisfaction_score"].mean().plot(kind="bar", color="#2E5B9A")
121     plt.title("Average Satisfaction Score by Region")
122     plt.ylabel("Avg Satisfaction (1-5)")
123     plt.tight_layout()
124     plt.savefig(f"{outdir}/satisfaction_by_region.png")
125     plt.close()
126
127     print(f"\nSaved charts to '{outdir}/'.")
128
129
130 def main():
131     df = load_data(INPUT_FILE)
132     df = clean_data(df)
133     df = add_features(df)
134     exploratory_summary(df)
135     identify_bottlenecks(df)
136     make_visualizations(df, OUTPUT_DIR)
137
138
139 if __name__ == "__main__":
    main()

```